

Spring Boot 3.0 Security

In Spring Boot 2.0 we provide security using **WebSecurityConfigurationAdapter**. It deprecated in Spring Boot in 3.0.

WebSecurityConfigurationAdapter class have 3 methods those we have to override.

This method is for Authentication and we can add any number of users.

```
public class EMSSecurityConfig extends WebSecurityConfigurationAdapter {

    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws Exception {
        auth.inMemoryAuthentication().inMemoryUserDetailsManagerConfigurer<AuthenticationManagerBuilder>
            .withUser( username: "Basant") UserDetailsManagerConfigurer<...>.UserDetailsBuilder
            .password(passwordEncoder().encode( rawPassword: "Pwd1"))
            .roles("USER");
        auth.inMemoryAuthentication().inMemoryUserDetailsManagerConfigurer<AuthenticationManagerBuilder>
            .withUser( username: "Santosh") UserDetailsManagerConfigurer<...>.UserDetailsBuilder
            .password(passwordEncoder()
                .encode( rawPassword: "Pwd2"))
            .roles("ADMIN");
    }
}
```

This method is for authorisation for endpoints

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.csrf().disable() HttpSecurity
        .authorizeRequests().antMatchers( ...antPatterns: "/text").permitAll() ExpressionUrlAuthorizationConfigurer<...>.Expre
        .and() HttpSecurity
        .authorizeRequests().antMatchers( ...antPatterns: "/greeting", "/text/**").authenticated() ExpressionUrlAuthori
        .and().httpBasic();
}
```

Dependency for security

<dependency>

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

```

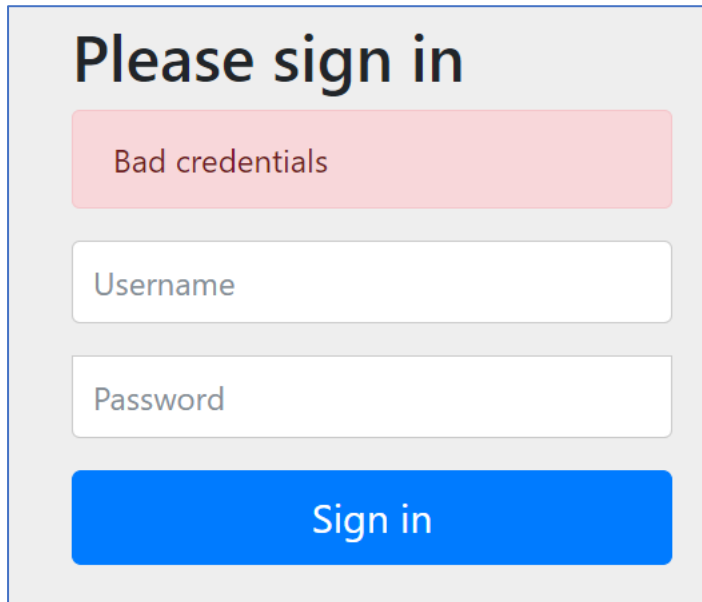
Application. Properties

```

spring.security.user.name=nagi
spring.security.user.password=123

```

if you provide incorrect credentials (bad credentials)



```

5 @Configuration
6 @EnableWebSecurity
7 public class SecurityConfig {
8
9
10     @Bean
11     //Authentication
12     public UserDetailsService userDetailsService(PasswordEncoder encoder) {
13
14         UserDetails admin =User.withUsername("nagi").password(encoder.encode("123")).roles("Admin").build();
15         UserDetails developer =User.withUsername("naveen").password(encoder.encode("456")).roles("Developer").build();
16         return new InMemoryUserDetailsManager(admin, developer);
17     }
18
19
20     @Bean
21     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
22         return http.csrf().disable().authorizeHttpRequests().requestMatchers("/product/welcome").permitAll()
23             .and().authorizeHttpRequests().requestMatchers("/products/**").authenticated().and().formLogin().and().build();
24     }
25
26
27     @Bean
28     public PasswordEncoder passwordEncoder() {
29
30         return new BCryptPasswordEncoder();
31     }
32
33

```

To add method level authorization:

```

15
16 @Configuration
17 @EnableWebSecurity
18 @EnableMethodSecurity
19 public class SecurityConfig {
20
21
22     @Bean
23     //Authentication
24     public UserDetailsService userDetailsService(PasswordEncoder encoder) {
25
26         UserDetails admin = User.withUsername("nagi").password(encoder.encode("123")).roles("Admin").build();
27         UserDetails developer = User.withUsername("naveen").password(encoder.encode("456")).roles("Developer").build();
28         return new InMemoryUserDetailsManager(admin, developer);
29     }
30
31
32     @Bean
33     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
34         return http.csrf().disable().authorizeHttpRequests().requestMatchers("/product/welcome").permitAll()
35             .and().authorizeHttpRequests().requestMatchers("/products/**").authenticated().and().formLogin().and().build();
36
37     }
38
39
40
41     @Bean
42     public PasswordEncoder passwordEncoder() {
43
44         return new BCryptPasswordEncoder();
45     }
46

```

```

7
8 @RestController
9 @RequestMapping("/products")
10 public class ProductController {
11
12     @GetMapping("/welcome")
13     public String welcome() {
14         return "Welcome this endpoint is not secure";
15     }
16
17
18     @GetMapping("/user")
19     @PreAuthorize("hasAuthority('ROLE_Developer')")
20     public String welcome1() {
21         return "Welcome this endpoint is password secure";
22     }
23
24     @GetMapping("/admin")
25     @PreAuthorize("hasAuthority('ROLE_Admin')")
26     public String welcome2() {
27         return "Welcome this endpoint is password secure";
28     }
29
30 }

```

JWT

JWT token contains Header, Payload and Signature.

<pre>{ "alg": "HS256", "typ": "JWT" }</pre>
PAYLOAD: DATA
<pre>{ "sub": "1234567890", "name": "Nagireddy", "iat": 1516239022 }</pre>
VERIFY SIGNATURE
<pre>HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), your-256-bit-secret) ☐ secret base64 encoded</pre>

To implement JWT, you need following dependencies in pom.xml

```
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-api</artifactId>
<version>0.11.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
```

1. JWT(Json Web Token) contains 3 parts. 1) Header 2) Payload and 3) Signature
Header: contains the algorithm and type of the token
Payload: contains actual username and expiry time of the token
Signature: How token encrypted (it is a combination of header, payload and secret).
2. Create an endpoint for all to generate token with DTO(username and password)
3. Create a service class to generate token

- Create a claims

```
Map<String,Object> claims = new HashMap<>();
```

```
public String generateToken(String userName){
    Map<String,Object> claims=new HashMap<>();
    return createToken(claims,userName);
}

private String createToken(Map<String, Object> claims, String userName) {
    return Jwts.builder()
        .setClaims(claims)
        .setSubject(userName)
        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis()+1000*60*30))
        .signWith(key: getSignKey())
    }
}
```

```
private String createToken(Map<String, Object> claims, String userName) {
    return Jwts.builder()
        .setClaims(claims)
        .setSubject(userName)
        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis()+1000*60*30))
        .signWith(getSignKey(), SignatureAlgorithm.HS256).compact();
    }
}
```

```
private Key getSignKey() {
    byte[] keyBytes= Decoders.BASE64.decode(SECRET);
    return Keys.hmacShaKeyFor(keyBytes);
}
```

4. Go to encryption key generator site and get the key with Hex format
(<https://www.allkeysgenerator.com/Random/Security-Encryption-Key-Generator.aspx>)
5. Call **generateToken()** method from controller by passing username
6. Update the **SecurityFilterChain** class to allow all users to generate token

The above code will create the token for all users even though, you provide incorrect username and passwords.

To prevent this:

1. Autowire AuthenticationManager in controller class and

```
3= @PostMapping("/authenticate")
4 public String authenticateAndGetToken(@RequestBody AuthRequest authRequest) {
5     Authentication authentication = authenticationManager.authenticate(
6         new UsernamePasswordAuthenticationToken(authRequest.getUsername(), authRequest.getPassword()));
7     if (authentication.isAuthenticated()) {
8         return jwtService.generateToken(authRequest.getUsername());
9     } else {
10        throw new UsernameNotFoundException("invalid user request !");
11    }
12 }
13 }
```

2. In Spring Boot 3.0, you need to explicitly define AuthenticationManager

```
75= @Bean
76 public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
77     return config.getAuthenticationManager();
78 }
79 }
```

How to validate Token?

1. Create filter class which extends from OncePerRequestFilter
2. @Override doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
 - Get the authHeader from request.
 - Get the token from the authHeader by removing prefix ("Bearer").
 - Extract the username from token
 - Extract claims from token by passing token and subject

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)
{
    String authHeader = request.getHeader("Authorization");
    String token = null;
    String username = null;
    if (authHeader != null && authHeader.startsWith("Bearer ")) {
        token = authHeader.substring(7);
        username = jwtService.extractUsername(token);
    }
}
```

```
25= public String extractUsername(String token) {
26     return extractClaim(token, Claims::getSubject);
27 }
28
29= public Date extractExpiration(String token) {
30     return extractClaim(token, Claims::getExpiration);
31 }
32
33= public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
34     final Claims claims = extractAllClaims(token);
35     return claimsResolver.apply(claims);
36 }
```

- Check the username !=null and SecurityContextHolder is null then
- Get the UserDetails from database
 - Validate the token by providing token and user details.
 - If valid then create UsernamePasswordAuthenticationToken and set SecurityContextHolder

➤ Else

Do filterChain.doFilter(request,response);

```

29  @Override
30  protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain) throws ServletException, IOException {
31      String authHeader = request.getHeader("Authorization");
32      String token = null;
33      String username = null;
34      if (authHeader != null && authHeader.startsWith("Bearer ")) {
35          token = authHeader.substring(7);
36          username = jwtService.extractUsername(token);
37      }
38
39      if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
40          UserDetails userDetails = userDetailsService.loadUserByUsername(username);
41          if (jwtService.validateToken(token, userDetails)) {
42              UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities());
43              authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
44              SecurityContextHolder.getContext().setAuthentication(authToken);
45          }
46      }
47      filterChain.doFilter(request, response);
48  }
49  }

```

3. Since above token is custom token. So we need change the SecurityFilterChain to validate our token as below

- Create a session management with securitypolicy as STATELESS
- Add authenticationProvider
- Add filter before
- And build it.

```

47  @Bean
48  public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
49      return http.csrf().disable()
50          .authorizeHttpRequests()
51              .requestMatchers("/products/new", "/products/authenticate").permitAll()
52              .and()
53              .authorizeHttpRequests().requestMatchers("/products/**")
54                  .authenticated().and()
55              .sessionManagement()
56                  .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
57                  .and()
58              .authenticationProvider(authenticationProvider())
59              .addFilterBefore(authFilter, UsernamePasswordAuthenticationFilter.class)
60              .build();
61  }

```

Q. How AuthenticationManager talk to DB?

In Config class

- You need UserDetailsService which will talk to DB
- @Override the method loadUserByUsername(String username)
- Create a CustomUserDetails class which implements from UserDetails interface
- @Override the all methods and properties.


```

public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private UserCredentialRepository repository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        Optional<UserCredential> credential = repository.findByName(username);
        return credential.map(CustomUserDetailsService::new).orElseThrow(() -> new UsernameNotFoundException("User not found"))
    }
}

```

```

@Bean
public UserDetailsService userDetailsService(){
    return new CustomUserDetailsService();
}

```

```

@Bean
public AuthenticationProvider authenticationProvider(){
    DaoAuthenticationProvider authenticationProvider=new DaoAuthenticationProvider();
    authenticationProvider.setUserDetailsService(userDetailsService());
    authenticationProvider.setPasswordEncoder(passwordEncoder());
    return authenticationProvider;
}

```